

ASIAN DEVELOPMENT FOUNDATION COLLEGE

P. Burgos St., Tacloban City

System Functional Integration Documentation

for LifeLink Emergency Blood-Donor Discovery and Coordination System

In Partial Fulfillment of the Requirements for the Subjects

System Integration & Architectures

Integrative Programming

Repository: `RamProjector/LifeLink`

Documentation date: October 1, 2026

Document status: Repository-based implementation documentation; replace bracketed fields and attach dated runtime evidence before submission.

Executive Summary

LifeLink is an authenticated emergency blood-donor discovery and coordination system. A native Kotlin/Jetpack Compose Android application captures a requester's blood need, urgency, approximate location, facility information, consent, and contact preference. The Android client sends an authenticated request to a PostgreSQL-backed FastAPI service. The service validates the request, applies rule-based blood compatibility and location matching, scores eligible donors, stores the emergency request and matches, and sends optional Firebase Cloud Messaging notifications. The client receives typed JSON responses, displays donor matches, lets the requester select donors, and creates controlled contact requests.

The system also supports donor onboarding, donor availability, donor GPS/location refresh, donor inboxes, donor accept/decline/arrival responses, request status polling, cancellation, fulfillment, offline draft and submission handling, profile capability persistence, push-token registration, account-scoped Room persistence, and privacy-safe distance-band map summaries.

The repository currently contains an in-progress privacy-first extension for **opt-in exact donor location sharing after donor acceptance** and **request-scoped in-app chat**. The intended

boundary is that exact donor pins are never public: they are returned only to the owner of an active emergency request when the donor has accepted the contact request, opted into exact location sharing, and has a fresh location snapshot. Chat is similarly authorized by the request/contact relationship. These new changes are documented here as implementation-in-progress until Android compilation, live database migration, and two-account end-to-end verification are completed.

The integration boundary is:

```
Android Kotlin/Compose client
  |
  | HTTPS + Supabase access token + JSON
  v
FastAPI PostgreSQL adapter on Render or local host
  |
  | async SQLAlchemy + ownership checks
  v
Supabase PostgreSQL/PostGIS database
  |
  +→ Firebase Cloud Messaging, optional push delivery
```

The strongest current automated evidence is the backend suite: **45 tests passed** on the inspected workspace after the donor freshness policy was implemented. Android validation is defined by the repository as Gradle debug build, unit tests, and lint; a live Android SDK/emulator run and hosted GitHub Actions result must be attached to the final school submission.

Document Guide

Section	Demonstrates
Chapter 1 — Introduction	Problem, scope, objectives, users, and safety boundary
Chapter 2 — System Architecture	Components, deployment, network path, database boundary, and data flow
Chapter 3 — Functional Integration Design	Requirements, workflows, contracts, authorization, privacy, and failure behavior
Chapter 4 — Implementation	Repository structure, representative code, migrations, Android integration, and deployment
Chapter 5 — Testing and Results	Automated checks, functional matrix, evidence rules, and unresolved verification
Chapter 6 — Conclusion and Recommendations	Achievements, limitations, and next steps
References	External technical and policy references used by the implementation
Appendix A	Environment and installation checklist
Appendix B	Evidence register for screenshots and logs
Appendix C	Group contribution record template
Appendix D	API and data-contract reference

Chapter 1 — Introduction

1.1 Background

Emergency blood requests require fast coordination among requesters, potential donors, facilities, and coordinators. A purely local mobile screen cannot reliably discover current donors

or preserve request state across accounts and devices. LifeLink centralizes matching and authorization in a backend service while keeping the user experience in a native Android client.

The system is intentionally a **coordination and discovery tool**, not a medical screening or blood-bank eligibility authority. Blood compatibility rules and matching scores help prioritize communication; they do not replace licensed facility screening, blood-bank confirmation, consent, or emergency medical services.

The repository is a cloud-ready package containing:

- `LifeLinkAndroid/` — native Kotlin/Jetpack Compose application;
- `lifelink_fastapi/` — FastAPI service, SQLAlchemy models, migrations, tests, and deployment files;
- `lifelink-mobile/` — Expo/React Native source retained as an additional mobile package;
- `docs/` — research, implementation status, deployment, testing, and project-history documents;
- `artifacts/` — APK and visual assets where present.

1.2 Problem Statement

A requester needs to submit an emergency blood request using a real location, discover nearby eligible donors, contact willing donors, and follow the request lifecycle without exposing private donor information to everyone. A donor needs to maintain availability, respond to eligible requests, and control whether an accepted requester may see a fresh exact location. The system must keep these operations consistent across Android UI, API, database, notifications, and offline persistence.

The integration problem is to make the following components complete one observable, authorized function:

1. Android requester/donor UI;
2. Android repositories, Room persistence, authentication, location, and notification code;
3. FastAPI request, donor, contact, location, and chat endpoints;
4. PostgreSQL/PostGIS schema and SQL migrations;
5. optional Render hosting, Supabase authentication/database, and Firebase notifications.

1.3 Objectives

1. Provide authenticated requester and donor onboarding without forcing a user into one exclusive role.
2. Capture approximate requester and donor locations using foreground, user-initiated location access.
3. Match compatible, available donors by distance, travel estimate, freshness, verification, response likelihood, and urgency.
4. Show requester-safe donor results without returning donor coordinates in ordinary match responses.
5. Allow a requester to select donors and send controlled contact requests.
6. Allow donors to accept, decline, or mark arrival, while preserving consent before contact disclosure.
7. Support explicit donor opt-in exact-location sharing only after an accepted contact relationship.
8. Support request-scoped in-app messages with per-request and per-participant authorization.
9. Preserve account ownership across local Room caches, offline workers, server requests, and notifications.
10. Provide repeatable automated and manual integration tests with clear failure behavior.

1.4 Scope and Limitations

In scope

- Supabase email/password authentication and access-token refresh;
- requester emergency-request wizard;
- donor profile and availability setup;
- native MapLibre map using OpenFreeMap Liberty style;
- GPS capture, manual coordinate fallback, location precision metadata, and freshness cutoff;
- PostgreSQL/PostGIS-compatible storage and SQL migrations;
- explainable donor matching and request persistence;
- controlled contact requests and lifecycle states;
- optional FCM push notifications and account-scoped token registration;

- offline draft/submission retry behavior;
- active-request polling, cancellation, manual broadcast fallback, and fulfillment;
- privacy-safe anonymous donor distance-band map;
- in-progress opt-in exact donor pins and request-scoped chat contracts.

Out of scope or not production-complete

- medical screening or blood-bank certification;
- guaranteed emergency response, dispatch, transportation, or clinical advice;
- public donor directory or public exact-location map;
- continuous/background donor GPS tracking;
- real-time WebSocket chat, typing indicators, read receipts, attachments, or calls;
- automatic phone-number disclosure without donor consent;
- production-grade moderation operations and abuse-response staffing;
- signed production Android release configuration;
- complete live deployment and two-account validation of the newest location/chat migration;
- crash analytics and full real-device accessibility/performance certification.

1.5 Intended Users

User	Primary responsibilities
Requester	Create a request, review matches, select donors, monitor status, communicate after acceptance, and close the request
Donor	Complete profile, set availability, review eligible requests, respond, optionally share exact location, and communicate after acceptance
Coordinator/facility	Review operational outcomes and perform licensed screening outside LifeLink
Administrator/developer	Configure authentication, database, API deployment, FCM, migrations, tests, and monitoring
Instructor/reviewer	Inspect architecture, integration contracts, source implementation, tests, and evidence

1.6 Safety and Privacy Boundary

LifeLink must fail closed for private resources. A donor's exact coordinates are not part of ordinary match results. The current privacy-first design returns exact pins only when the donor opted in, the requester owns the active request, the donor is matched to that request, the donor accepted the contact relationship, the request is non-terminal, and the location is fresh. Chat uses the same request/contact relationship and is not addressable by an arbitrary donor or request ID alone.

Chapter 2 — System Architecture

2.1 Component Placement

Component	Location	Responsibility
Android Compose client	User Android device	Authentication, requester/donor UI, location capture, maps, local persistence, API calls, notification display
Android Room database	User Android device	Account-scoped drafts, pending submissions, active requests, donor profiles/inboxes, and updates
FastAPI application	Render, local host, or other server	HTTP API, validation, authentication dependency, ownership checks, matching, lifecycle, contact, location, chat, push orchestration
SQLAlchemy async adapter	FastAPI process	Maps typed API operations to PostgreSQL rows and transactions
PostgreSQL/PostGIS	Supabase or local PostgreSQL	Users/profiles, facilities, requests, matches, donor state, contacts, audit events, location metadata, conversations, and messages
Supabase Auth	Hosted identity provider	Email/password accounts, JWT access tokens, refresh sessions, user subjects
Firebase Cloud Messaging	Optional external service	Donor match, contact request, donor response, and other targeted push notifications
Render configuration	<code>render.yaml</code> and Dockerfile	Public API deployment, environment variables, health checks, and free-tier hosting configuration
GitHub Actions	Repository CI	Backend tests, Android build/unit/lint, visual review, and protected database diagnostics where configured

2.2 Logical Architecture

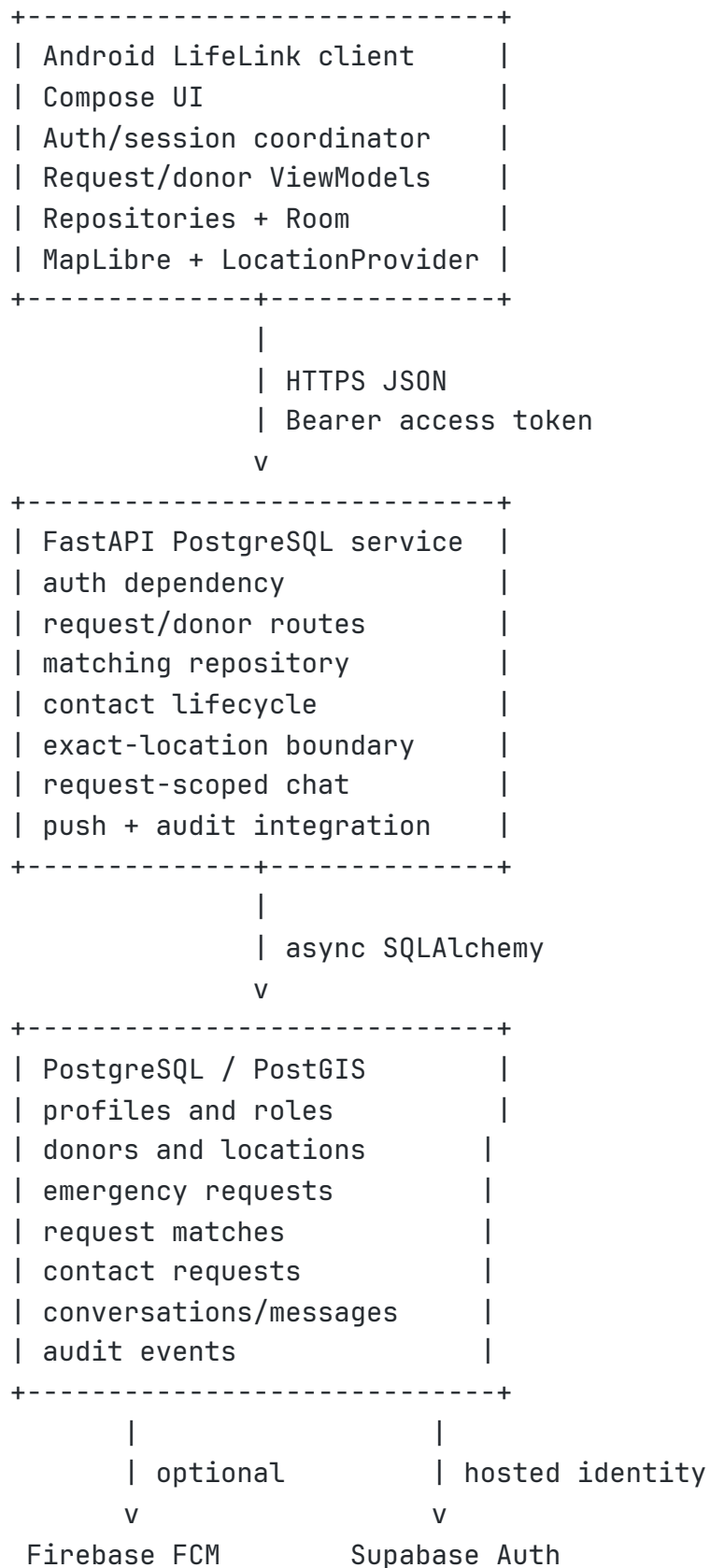


Figure 1. Logical component and data path of LifeLink.

2.3 Deployment Configuration

Local backend

```
API:      http://localhost:8000/
Docs:     http://localhost:8000/docs
Demo app: app.main:app
Cloud app: app.main_postgres:app
Database: PostgreSQL or local test database
```

Android emulator against local backend

```
Android emulator API base URL: http://10.0.2.2:8000/
```

Cleartext HTTP is permitted only for the emulator address in the Android configuration. A physical device should use a reachable HTTPS development or deployed API URL.

Hosted MVP architecture

```
Android app → Render Free FastAPI → Supabase Free PostgreSQL/PostGIS
|
+→ Firebase Cloud Messaging, optional
```

The documented public Render API is <https://lifelink-api-uzje.onrender.com/>. The endpoint should be rechecked before final submission because hosted free services can sleep and cold-start. No private credentials are included in this document.

Required environment variables

Variable	Purpose
<code>LIFELINK_DATABASE_URL</code>	PostgreSQL connection string; hosted <code>sslmode=require</code> is translated for asyncpg
<code>LIFELINK_AUTH_REQUIRED</code>	Fail-closed authentication mode for deployed environments
<code>LIFELINK_DONOR_LOCATION_MAX_AGE_MINUTES</code>	Freshness cutoff; documented default is 1440 minutes
<code>DB_POOL_SIZE</code> / <code>DB_MAX_OVERFLOW</code>	Async SQLAlchemy pool sizing
<code>FIREBASE_SERVICE_ACCOUNT_JSON</code>	Optional server credential for FCM delivery; never commit it
<code>LIFELINK_API_BASE_URL</code>	Android API base URL override
<code>SUPABASE_URL</code>	Android Supabase project URL
<code>SUPABASE_PUBLISHABLE_KEY</code>	Android publishable client key; never use a service-role key in the app

2.4 Database Boundary

The Android app never connects directly to PostgreSQL. The FastAPI service owns database credentials, validation, authorization, matching, contact operations, and migration compatibility. The current ordered SQL migrations include the initial schema, GPS request location, roles/profiles/contacts, contact lifecycle, audit events, donor operational profile, and the in-progress location-sharing/chat migration.

Core relational entities are:

Entity	Important fields/relationships
<code>lifelink_profiles</code>	Auth subject, email, display name, request/donor capabilities, FCM token
<code>donors</code>	User identity, blood type, coordinates, precision, availability, visibility, service radius, freshness, location-sharing preference
<code>facilities</code>	Verified facility identity, name, area, and optional coordinates
<code>emergency_requests</code>	Requester, need, urgency, deadline, approximate location, facility, consent, status
<code>request_matches</code>	Request/donor pair, rank, score, distance, travel estimate, explanation, response state
<code>donor_contact_requests</code>	Requester/donor relationship, lifecycle status, consent timestamps, contact-sharing timestamp
<code>audit_events</code>	Append-only actor/action/request/donor metadata for operational traceability
<code>conversations</code>	One request-scoped requester/donor conversation, unique per request/donor pair
<code>conversation_messages</code>	Sender, body, timestamp, foreign key to conversation

2.5 Data Flow: Request to Donor Contact

1. The requester authenticates and the Android client obtains an access token.
2. The requester completes the wizard and selects a facility or approximate GPS location.
3. The client submits `POST /v1/emergency-requests` with an idempotency key.
4. FastAPI verifies identity and requester ownership, validates fields, excludes self-matches, filters eligible donors, and scores matches.
5. The request, matches, explanation, and status are committed to PostgreSQL.
6. Optional FCM notifications target the matched donor accounts.

7. The requester receives typed match results containing donor identity, distance, travel estimate, and explanation — not donor coordinates.
8. The requester selects donor IDs and submits `POST /v1/emergency-requests/{request_id}/contact` .
9. The server rechecks request ownership and match eligibility, persists contact requests, and notifies donors.
10. The donor accepts/declines/marks arrival through the donor endpoint.
11. After acceptance, requester contact details may be disclosed according to the contact lifecycle; exact donor pins and chat are independently authorized by the new privacy boundary.

2.6 Account and Offline Data Flow

All account-specific Room queries are scoped by authenticated user ID. Logout clears local account data and schedules durable cleanup. Offline drafts remain local; offline submissions carry both owner ID and exact draft ID in their WorkManager input. A queued submission is retried only when network constraints are met and is not selected by a global “oldest request” query.

Chapter 3 — Functional Integration Design

3.1 Functional Requirements and Acceptance Criteria

FR-01 — Authenticated account and capabilities

A user can sign in, refresh an expired access token, maintain requester capability, and optionally complete donor setup without choosing an exclusive role. Server profile capability values must not be silently reverted.

Acceptance: a new account reaches requester home; an existing donor-capable account restores its server profile; a user with both capabilities can enter donor mode; signing out and changing accounts does not show prior-account data.

FR-02 — Request creation and matching

A requester can submit blood type, units, urgency, deadline, facility/location, note, contact method, consent, and matching preference. The server validates and ranks eligible donors.

Acceptance: valid requests return a request ID and ranked matches; unknown blood type returns manual fallback; self-donor matches are excluded; duplicate retries with the same idempotency key do not create duplicate requests.

FR-03 — Donor onboarding and availability

A donor can save display name, blood type, approximate location, service radius, note, preferred contact method, visibility, pause reason, and availability.

Acceptance: incomplete profiles cannot select active availability or receive donor opportunities; saved server profile is restored before donor inbox loading; donors can refresh location/availability.

FR-04 — Controlled donor contact

A requester can select eligible matches and create contact requests. A donor can accept, decline, or mark arrival. Contact lifecycle changes are persisted and visible to the correct participants.

Acceptance: requester sees pending/accepted/declined/arrived/later lifecycle states; donor cannot respond to their own request; stale client selections are rejected server-side; email/contact details are not visible before the permitted state.

FR-05 — Privacy-safe map behavior

The ordinary requester map shows only the requester location and anonymous distance-band counts. It must not show donor names or exact donor pins.

Acceptance: no donor coordinates are present in ordinary `DonorMatchResponse` ; no donor pin appears in the public/anonymous map; a missing requester location does not render `(0,0)` as if it were real.

FR-06 — Opt-in exact donor location

A donor can explicitly enable “Share exact location after acceptance.” The requester can fetch exact donor pins only for accepted contacts in an active request when the donor’s location is fresh.

Acceptance: sharing is off by default; disabled sharing returns no exact pin; pending/declined contacts return no exact pin; non-requester accounts receive `403` ; terminal requests return no locations; stale locations are excluded; accepted requester can see the donor pin and timestamp.

FR-07 — Request-scoped chat

After donor acceptance, the requester and donor can read and send messages in the conversation tied to one emergency request and one donor.

Acceptance: both participants can read/send; a nonparticipant receives 403 ; pending contacts cannot chat; terminal requests cannot chat; message body is trimmed and limited to 2,000 characters; messages persist in order.

FR-08 — Notifications and deep ownership

FCM messages are targeted with authenticated recipient identity. Android displays a push only when its destination matches the signed-in user.

Acceptance: a targeted push is stored/displayed only for its intended account; untargeted or mismatched pushes are ignored; missing Firebase credentials fail safely without breaking local development.

FR-09 — Failure recovery

The app presents user-safe validation, authentication, network, timeout, server, and empty-response messages. It does not display raw SQL, secrets, or stale donor/request data after a failed read.

Acceptance: backend tests cover invalid/missing records, lifecycle conflicts, auth/ownership cases, and stale matching; Android tests cover Room migrations, offline ownership, refresh behavior, and authentication races.

3.2 Primary Workflows

Workflow A — Requester creates and submits a request

1. User opens the emergency request wizard.
2. User selects blood type or unknown type, units, urgency, deadline, facility/location, note, and contact method.
3. Android requests foreground coarse/fine location only when the user captures a location; manual map/coordinate entry remains available.
4. User confirms genuine request and sharing consent.
5. ViewModel validates the draft, saves it locally, and submits it with an idempotency key.
6. FastAPI authenticates the subject and checks that the requester ID matches.

7. Matching filters self-requests, availability, visibility, freshness, blood compatibility, radius, and optional verification policy.
8. Server stores the request and returns ranked matches or manual fallback.
9. Android opens the results state and refreshes contact activity.

Workflow B — Requester contacts and chats with a donor

1. Requester selects one or more donor cards.
2. Android sends donor IDs to the contact endpoint.
3. Server checks request ownership, active status, and match membership.
4. Donor receives an optional push and sees the request in the donor inbox.
5. Donor accepts.
6. Requester refreshes contact activity and may request authorized contact details.
7. If the donor opted into exact location sharing, requester fetches authorized pins.
8. Requester and donor use the request-scoped chat endpoints.
9. Requester may arrange a meeting, fulfill, cancel, report, or block according to lifecycle rules.

Workflow C — Donor updates location sharing

1. Donor opens donor profile settings.
2. Donor sees the default-off exact-location sharing switch.
3. Donor enables or disables the preference and saves the profile.
4. Android sends the preference with the donor profile payload.
5. Backend persists `location_sharing_enabled`.
6. The exact location endpoint evaluates the preference at read time; disabling it removes the donor from authorized pin results.

3.3 Public API Contract

Authentication and profiles

```
GET    /v1/profile
PUT    /v1/profile
PUT    /v1/push-token
```

The client sends a bearer access token. The server derives the authenticated subject and does not trust a client-provided user ID for ownership decisions.

Requester request lifecycle

```
POST    /v1/emergency-requests
GET     /v1/emergency-requests
GET     /v1/emergency-requests/{request_id}
POST    /v1/emergency-requests/{request_id}/manual-broadcast
POST    /v1/emergency-requests/{request_id}/contact
GET     /v1/emergency-requests/{request_id}/contacts
PATCH  /v1/emergency-requests/{request_id}/contacts/{donor_id}
POST    /v1/emergency-requests/{request_id}/contacts/{donor_id}/report
POST    /v1/emergency-requests/{request_id}/contacts/{donor_id}/block
POST    /v1/emergency-requests/{request_id}/cancel
POST    /v1/emergency-requests/{request_id}/fulfill
```

Donor operations

```
PUT     /v1/donors/{donor_id}
GET     /v1/donors/{donor_id}
PATCH  /v1/donors/{donor_id}/availability
GET     /v1/donors/{donor_id}/requests
POST    /v1/donors/{donor_id}/requests/{request_id}/response
```

Privacy-first extension

```
GET /v1/emergency-requests/{request_id}/donor-locations
GET /v1/emergency-requests/{request_id}/conversations/{donor_id}/message:
POST /v1/emergency-requests/{request_id}/conversations/{donor_id}/message:
```

3.4 Representative JSON Contracts

Create request

```
{
  "requester_id": "authenticated-user-subject",
  "blood_type": "O-",
  "units": 1,
  "urgency": "urgent",
  "response_deadline": "2026-10-01T12:30:00Z",
  "location": {
    "facility_id": "facility-001",
    "facility_name": "Verified Facility",
    "area": "Tacloban City",
    "latitude": 11.2449,
    "longitude": 125.0030,
    "precision_meters": 100,
    "verified": true
  },
  "contact_method": "in_app",
  "note": "Need one unit; facility will screen donor.",
  "genuine_request_confirmed": true,
  "sharing_consent_confirmed": true,
  "ai_matching_enabled": true,
  "idempotency_key": "draft-uuid"
}
```

Donor match response

```
{
  "donor_id": "donor-001",
  "display_name": "Donor Display Name",
  "blood_type": "O-",
  "distance_km": 4.2,
  "estimated_travel_minutes": 18,
  "score": 0.87,
  "explanation": {
    "eligible": true,
    "factors": ["Compatible blood type", "Within service radius", "Freshness"],
    "score_breakdown": {
      "distance": 0.9,
      "freshness": 1.0,
      "urgency": 0.8
    }
  }
}
```

Important: this response intentionally contains no donor latitude or longitude.

Authorized location response

```
[
  {
    "donor_id": "donor-001",
    "display_name": "Donor Display Name",
    "latitude": 11.2500,
    "longitude": 125.0100,
    "availability_updated_at": "2026-10-01T02:00:00Z"
  }
]
```

This response is returned only after the endpoint performs request ownership, accepted-contact, donor opt-in, active-request, and freshness checks.

Chat message

```
{
  "id": "message-uuid",
  "sender_id": "authenticated-user-subject",
  "body": "I am at the facility entrance.",
  "created_at": "2026-10-01T02:05:00Z"
}
```

3.5 Error and State Handling

Condition	Expected visible behavior	Technical handling
Blank/invalid request field	Inline validation message	Android blocks submit; server validates again
Unknown blood type	Manual broadcast explanation	Server stores manual fallback; no automatic match decision
Requester not owner	Request unavailable/not allowed	Server returns <code>403</code> ; client does not reveal private data
Donor pending contact	Waiting for donor response	Exact pin/chat endpoints return <code>403</code>
Donor declined	No contact details or pin	Contact lifecycle remains declined
Donor opted out	No exact pin	Query requires <code>location_sharing_enabled = true</code>
Stale donor location	Donor omitted from exact-pin response and new matching	Freshness cutoff uses <code>LIFELINK_DONOR_LOCATION_MAX_AGE_MINUTES</code>
Terminal request	No chat/location actions	Server rejects or returns empty authorized location list
Offline	Draft saved; retry message	Room and WorkManager preserve owner/draft identity
Expired access token	Refresh/retry or sign-in prompt	Auth coordinator refreshes token once and preserves safe state
API unavailable	User-safe retry message	Repository maps network/timeout failure; no raw details

Condition	Expected visible behavior	Technical handling
Empty contact response	Uncertain status shown	Client refreshes contact activity before retrying to avoid duplicates
FCM destination mismatch	Notification ignored	Android checks authenticated recipient ID

3.6 Authorization Rules

Authorization is applied to every private request, not only at screen entry:

- requesters can read/manage only their own emergency requests;
- donors can read only eligible donor inbox items for their authenticated donor identity;
- donor responses require a match and reject self-request cases;
- contact reads/writes require the request/donor relationship;
- exact donor locations require requester ownership plus accepted contact plus donor opt-in plus freshness;
- chat reads/writes require accepted contact plus requester/donor participant identity;
- terminal requests disable sensitive location/chat operations;
- local caches and push histories are account-scoped.

Chapter 4 — Implementation

4.1 Solution Structure

```
LifeLink/
├── LifeLinkAndroid/
│   ├── app/src/main/java/com/lifelink/app/
│   │   ├── core/auth/           Supabase session and token refresh
│   │   ├── core/location/       LocationProvider and MapLibre maps
│   │   ├── core/notifications/  FCM token and message handling
│   │   ├── data/local/          Room entities, DAOs, workers, migr
│   │   ├── data/remote/         Retrofit API and JSON models
│   │   ├── data/repository/     Account-bound repositories
│   │   ├── domain/              Typed app contracts and state mode
│   │   └── feature/             Compose screens and ViewModels
│   ├── app/src/test/            JVM/Robolectric/unit tests
│   └── README.md                Android build and validation guide
├── lifelink_fastapi/
│   ├── app/main.py              In-memory demo service and models
│   ├── app/main_postgres.py     PostgreSQL FastAPI adapter
│   ├── app/db_models.py         SQLAlchemy typed models
│   ├── app/repositories.py      Request/donor persistence adapter
│   ├── app/donor_repositories.py Donor profile/inbox/response adapt
│   ├── app/location_chat_routes.py In-progress authorized pins/chat rou
│   ├── app/security.py          Principal/auth dependency
│   ├── sql/                     Ordered schema migrations
│   ├── tests/                   Backend and PostgreSQL regression
│   ├── Dockerfile               Container deployment
│   └── render.yaml              Render environment template
├── docs/                        Research, status, deployment, and
├── .github/workflows/           CI and protected diagnostics
└── artifacts/                   APKs and visual assets where prese
```

4.2 Android Implementation

UI and state

The Android application uses Compose screens with ViewModels and typed `UiState` data classes. The requester flow is represented by `RequestStep` values: blood need, urgency, location, contact, review, and results. The donor flow separates profile setup from availability and restores the server profile before loading donor requests.

Network client

`LifeLinkApi.kt` defines Retrofit methods for profiles, push tokens, emergency requests, contacts, donors, authorized donor locations, and chat. JSON names use `@SerializedName` for snake_case API fields. Repositories convert remote responses into domain models and map failures to user-safe exceptions/messages.

Local persistence

Room database version 9 is the last committed baseline. The in-progress work bumps the database to version 10 and adds `locationSharingEnabled` to `donor_profiles` through a non-destructive migration. All queries remain owner-scoped. The migration must be exercised by Android tests before merge.

Location and map

`LocationProvider` uses foreground one-shot location capture with device settings resolution and precision metadata. `MapLibreLocationPicker` supports map selection and manual fallback. `MapLibrePrivacySafeDonorMap` shows the requester location and anonymous bands. The in-progress `MapLibreAuthorizedDonorMap` renders exact pins only from the authorized server response and is not suitable for public donor discovery.

Notifications

The Firebase messaging service checks message destination identity before displaying or persisting notifications. Push token registration is scheduled and serialized with account cleanup to avoid cross-account tokens.

4.3 Backend Implementation

Request/matching service

`main_postgres.py` exposes the production adapter. It uses `get_postgres_principal` for authenticated identity, `SqlAlchemyDonorRepository` for donor selection, and `SqlAlchemyRequestStore` for persistence. Matching includes compatibility, distance, service radius, availability, freshness, verification policy, response probability, and urgency.

Freshness policy

The backend reads `LIFELINK_DONOR_LOCATION_MAX_AGE_MINUTES`, defaulting to 1440 minutes. Donors beyond that age are excluded from new matching rather than receiving a lower score. The policy is documented in backend README and Render configuration.

Contact lifecycle

The request store persists contact requests, donor responses, requester status updates, contact-sharing timestamps, reports, and blocks. Email visibility is controlled by lifecycle state. Exact location and chat are separate permissions and do not automatically follow email disclosure.

In-progress location/chat adapter

`location_chat_routes.py` adds:

- `GET /v1/emergency-requests/{request_id}/donor-locations` ;
- `GET /v1/emergency-requests/{request_id}/conversations/{donor_id}/messages` ;
- `POST /v1/emergency-requests/{request_id}/conversations/{donor_id}/messages` .

`sql/007_location_sharing_and_chat.sql` adds the donor preference, `conversations`, and `conversation_messages` tables. The adapter checks the request owner, donor participant, accepted contact status, active request state, donor opt-in, and freshness. This code requires PostgreSQL migration and full integration verification before production use.

4.4 Representative Backend Authorization Code

```
async def _contact_for_user(session, request_id, donor_id, principal):
    result = await session.execute(
        select(DonorContactRequest, EmergencyRequest, Donor)
        .join(EmergencyRequest,
              EmergencyRequest.id == DonorContactRequest.request_id)
        .join(Donor, Donor.id == DonorContactRequest.donor_id)
        .where(
            DonorContactRequest.request_id == request_id,
            DonorContactRequest.donor_id == donor_id,
        )
    )
    row = result.one_or_none()
    if row is None:
        raise HTTPException(404, "Accepted contact not found")

    contact, request, donor = row
    donor_owner = donor.user_id or donor.id
    if principal.subject not in {
        request.requester_id, donor_owner, donor.id
    }:
        raise HTTPException(403, "This conversation is not available")
    if contact.status not in ACCEPTED_CONTACT_STATUSES:
        raise HTTPException(403, "Chat is available after acceptance")
    if request.status in TERMINAL_REQUEST_STATUSES:
        raise HTTPException(403, "This request is no longer active")
    return contact, request, donor
```

The exact-location route adds the requester-owner check, donor opt-in condition, and freshness cutoff before returning coordinates.

4.5 Representative Android Contract

```
data class AuthorizedDonorLocation(  
    val donorId: String,  
    val displayName: String,  
    val latitude: Double,  
    val longitude: Double,  
    val availabilityUpdatedAt: String  
)  
  
interface EmergencyRequestRepository {  
    suspend fun refreshAuthorizedDonorLocations(  
        requestId: String  
    ): List<AuthorizedDonorLocation>  
  
    suspend fun loadChatMessages(  
        requestId: String,  
        donorId: String  
    ): List<ChatMessage>  
  
    suspend fun sendChatMessage(  
        requestId: String,  
        donorId: String,  
        body: String  
    ): ChatMessage  
}
```

The Android client must treat an empty authorized-location response as normal privacy behavior, not as evidence that the donor is unavailable or that the map should show a fallback exact coordinate.

4.6 Deployment Sequence

1. Create or select a Supabase PostgreSQL project and enable PostGIS.
2. Apply SQL files in filename order to a new, empty database. The in-progress location/chat migration is `007_location_sharing_and_chat.sql`.
3. Configure the API with `LIFELINK_DATABASE_URL`, `LIFELINK_AUTH_REQUIRED=true`, freshness cutoff, pool sizes, and optional Firebase credentials.

4. Deploy FastAPI using the included Dockerfile/Render configuration.
5. Verify `/health` and `/openapi.json` without exposing secrets.
6. Configure Android API base URL, Supabase URL, and publishable key.
7. Build the debug APK and run Android unit tests/lint.
8. Install on two test devices/accounts: one requester, one donor.
9. Run contact acceptance, opt-in exact pin, opt-out, stale-location, chat participant, and terminal-request tests.
10. Capture masked evidence and record commit, deployment URL, migration state, date, and result.

Chapter 5 — Testing and Results

5.1 Test Setup and Evidence Rule

The repository was inspected in the Sandbox workspace at `/home/ubuntu/LifeLink`. The backend dependencies were installed and the following command completed successfully after the donor freshness implementation:

```
cd lifelink_fastapi
python3 -m compileall -q app tests
python3 -m pytest -q -rs
```

Observed result: `45 passed in 3.94s`.

This is repository evidence, not proof of a deployed production run. A final submission must add dated screenshots or logs from the Android app, FastAPI service, and database where appropriate. Mask email addresses, coordinates, tokens, patient details, and database credentials.

For every test case, record the date, environment, account type, input, expected result, actual result, evidence ID, and Pass/Fail status. Failed tests must include cause, correction, and retest.

5.2 Automated Validation Matrix

ID	Test/validation	Expected result	Current evidence/status
AT-01	Python compilation	No syntax/import compilation errors	Pass: compileall completed
AT-02	Backend pytest suite	Existing backend regressions remain green	Pass: 45 passed
AT-03	Git diff check	No whitespace errors in intended patch	Run before PR merge
AT-04	Android debug build	APK assembles with current Kotlin/Room/API changes	Required; attach CI result
AT-05	Android unit tests	Room/auth/repository tests pass	Required; attach CI result
AT-06	Android lint	No new lint blockers	Required; attach CI result
AT-07	PostgreSQL migration 007	Donor preference and chat tables apply cleanly	Required against disposable database
AT-08	PostgreSQL end-to-end	Authorized routes pass owner/relationship checks	Required; add regression tests

5.3 Functional Test Matrix

ID	Scenario	Expected result	Actual/evidence to record	Status
TC-01	New requester signs up	Requester shell opens without mandatory role choice	Device screenshot + auth/API log	Pending live evidence
TC-02	Existing donor signs in	Server donor profile restores before inbox	Device screenshot + API response	Pending live evidence
TC-03	Valid emergency request	Request ID and ranked matches returned	Masked request/result screenshot	Pending live evidence
TC-04	Unknown blood type	Manual fallback shown; no automatic compatibility decision	Result screenshot + server log	Pending live evidence
TC-05	Self-request attempt	Server rejects donor response/contact	API status and test log	Pending live evidence
TC-06	Offline submission	Draft/queue saved with owner and draft ID	Room/WorkManager evidence	Pending live evidence
TC-07	Accepted contact	Contact lifecycle shows accepted	Requester and donor screenshots	Pending live evidence
TC-08	Pending contact exact map	No exact donor pin; safe explanation shown	Requester screenshot + API 403	Pending live evidence
TC-09	Donor opt-in exact pin	Accepted requester sees fresh authorized pin	Two-account screenshots + API log	Pending live evidence

ID	Scenario	Expected result	Actual/evidence to record	Status
TC-10	Donor opts out	Previously visible pin disappears after refresh	Before/after screenshots	Pending live evidence
TC-11	Stale donor location	Pin omitted; donor excluded from new match	Database timestamp + API result	Pending live evidence
TC-12	Nonparticipant chat read	Server returns 403 and no message body	API test output	Pending live evidence
TC-13	Accepted chat send/read	Both participants see persisted message in order	Two-device screenshots	Pending live evidence
TC-14	Terminal request chat	Server blocks new read/write	API status and UI message	Pending live evidence
TC-15	Wrong FCM destination	Android ignores message	Notification log/screenshot	Pending live evidence
TC-16	Server stopped	User-safe retry message; no crash or stale result	Offline screenshot	Pending live evidence

5.4 Detailed Test Record — Exact Location

Field	Entry
Test ID	TC-09
Preconditions	PostgreSQL migration 007 applied; requester owns active request; donor is a ranked match; donor contact is accepted; donor profile is saved with sharing enabled; location is fresh
Steps	1. Sign in as donor. 2. Enable exact-location sharing and save. 3. Refresh donor location/availability. 4. Sign in as requester. 5. Open accepted contact. 6. Open map and refresh authorized locations.
Expected	A pin with donor display name and fresh timestamp appears only for the accepted requester. Ordinary match response still contains no coordinates.
Actual	Record observed result, HTTP status, commit, and screenshot IDs.
Evidence	E09 requester map; E10 donor setting; E11 masked API response; E12 server/database timestamp log
Result	Replace with actual dated Pass/Fail after live two-account run.

5.5 Detailed Test Record — Unauthorized Chat

Field	Entry
Test ID	TC-12
Preconditions	Conversation exists for requester A and donor B; third account C is authenticated; contact is accepted only between A and B
Steps	1. Authenticate as C. 2. Request A/B conversation URL. 3. Attempt GET and POST.
Expected	Both operations return 403 ; no message body or conversation metadata is disclosed beyond a safe error.
Actual	Record HTTP response and server log correlation ID.
Evidence	E13 API test output; E14 sanitized server log
Result	Replace with actual dated Pass/Fail.

5.6 Issue and Correction Log

Issue	Cause	Correction/current action	Retest
Donor map originally showed only anonymous bands	Privacy boundary did not yet include authorized exact pins	Added opt-in donor preference and accepted-contact exact-location route	TC-08 to TC-11
Continuous background tracking would increase privacy and policy risk	Product request was ambiguous about GPS behavior	Chosen design is foreground/user-initiated refresh; no background tracking	Permission review + TC-09
In-app chat was not previously implemented	Contact flow ended at controlled contact details	Added request-scoped conversation/message schema and endpoints	TC-12 to TC-14
Local Room schema changes can break existing installs	New donor preference needs persistence	Added version 10 migration	AT-04/05 and migration test
Backend tests do not prove new PostgreSQL route behavior	Existing 45-test suite predates the new route slice	Add route authorization and migration integration tests before merge	AT-07/08
Live exact-location behavior is not confirmed	Requires two authenticated accounts and deployed migration	Run two-device or emulator test and attach masked evidence	TC-09 to TC-14

5.7 Results Interpretation

The backend baseline is currently regression-green with 45 passing tests. The repository clearly demonstrates a working Android/FastAPI/PostgreSQL integration for the established requester/donor/contact flows. The exact-location/chat extension is architecturally aligned with

the privacy boundary and has been wired through schema, backend route, Android models, repositories, and donor UI, but it must not be described as production-complete until the Android build, migration, route tests, and live two-account workflow pass.

Chapter 6 — Conclusion and Recommendations

LifeLink integrates a native Android client, FastAPI service, PostgreSQL/PostGIS data layer, Supabase authentication, optional Firebase notifications, and account-scoped offline persistence into one emergency blood-donor coordination workflow. The system's strongest design decisions are server-owned authorization, explicit requester/donor capabilities, explainable matching, freshness enforcement, controlled contact lifecycle, safe failure behavior, and privacy protection against public donor-coordinate disclosure.

The current repository proves the backend baseline through a passing 45-test suite and contains deployment, migration, Android build, and CI guidance. It also records the next product direction: donors may willingly share exact locations on a map, but only after accepting a requester's contact request and enabling the preference. Requester/donor chat is request-scoped and authorization-protected rather than a public messenger directory.

Before public or production use, the team should:

1. Add and pass dedicated PostgreSQL route tests for exact locations and chat.
2. Apply and verify migration 007 on a disposable and then the intended deployment database.
3. Complete Android Gradle build, unit tests, and lint after the Room version 10 change.
4. Run a two-account end-to-end workflow on real devices or emulators.
5. Add durable moderation, abuse reporting, rate-limit verification, and audit review for chat/location access.
6. Keep exact pins opt-in, fresh, requester-scoped, and disabled when a request becomes terminal.
7. Keep continuous/background tracking out of the MVP unless a new privacy, Android, Play-policy, battery, and consent review is completed.
8. Configure a real production signing key, crash reporting, HTTPS-only deployment, and secret rotation.
9. Validate blood compatibility and emergency operational rules with licensed local experts.

10. Replace polling with real-time delivery only after the authorization model and moderation workflow are proven.

References

1. Android Developers. **Request location permissions.**
<https://developer.android.com/develop/sensors-and-location/location/permissions>
2. Google Play. **Minimum Scope: Foreground Location Access and the Location Button.**
<https://support.google.com/googleplay/android-developer/answer/17033915?hl=en>
3. OWASP. **Authorization Cheat Sheet.**
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
4. OWASP. **A01:2021 Broken Access Control.** https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/
5. MapLibre Android documentation and OpenFreeMap Liberty style configuration used by the repository.
6. Supabase documentation for PostgreSQL, PostGIS, and authentication configuration.
7. Render documentation for free web-service deployment and cold-start behavior.
8. Repository documents: `README.md` , `LifeLinkAndroid/README.md` , `lifelink_fastapi/README.md` , `docs/IMPLEMENTATION_STATUS.md` , `docs/PROJECT_CONTEXT_AND_REGRESSION_GUARDRAILS.md` , and `docs/LOCATION_SHARING_AND_CHAT_RESEARCH_PLAN.md` .

Appendix A — Environment and Installation Checklist

- ☐ Confirm repository commit, branch, and documentation date.
- ☐ Install Java 21, Android SDK API 35, and use the included Gradle wrapper.
- ☐ Create a Python virtual environment and install `lifelink_fastapi/requirements.txt` .
- ☐ Create a disposable PostgreSQL database with PostGIS enabled.
- ☐ Apply SQL migrations in filename order, including migration 007 for the in-progress extension.

- ☐ Configure `LIFELINK_DATABASE_URL` without committing credentials.
- ☐ Configure `LIFELINK_AUTH_REQUIRED=true` for a deployed environment.
- ☐ Configure `LIFELINK_DONOR_LOCATION_MAX_AGE_MINUTES=1440` or an approved value.
- ☐ Configure Supabase URL and publishable key in Android Gradle properties/environment.
- ☐ Configure Firebase service credentials only in the server secret store if push delivery is tested.
- ☐ Run `/health` and `/openapi.json` checks.
- ☐ Run backend compile and pytest.
- ☐ Run Android assembleDebug, testDebugUnitTest, and lintDebug.
- ☐ Test one requester account and one donor account.
- ☐ Capture masked screenshots and logs for Appendix B.
- ☐ Confirm no private coordinates, tokens, passwords, patient details, or service-role keys appear in evidence.

Appendix B — Evidence Register

Evidence ID	Required image or log	Caption to write
E01	Android requester home	Authenticated requester shell opens
E02	Android donor home/profile	Donor profile restored from server
E03	Requester request form	Valid blood request and location consent
E04	Requester results	Ranked donors with distance, no donor coordinates
E05	Anonymous map	Distance bands/counts only; no donor names/pins
E06	Donor inbox	Eligible request visible to donor
E07	Accepted contact	Donor acceptance and requester contact activity
E08	Backend test output	45 passed baseline regression suite
E09	Donor sharing switch	Exact sharing is explicit and default-off
E10	Authorized requester map	Accepted requester sees fresh authorized donor pin
E11	Disabled sharing result	Opted-out donor omitted from exact-pin response
E12	Stale location result	Freshness cutoff omits stale location
E13	Chat conversation	Requester and donor exchange a persisted message
E14	Unauthorized chat attempt	Nonparticipant receives 403 without message disclosure
E15	Offline/error state	User-safe retry message; no crash or stale data
E16	CI/build reports	Android build, unit test, lint, and migration evidence

Appendix C — Group Contribution Record

Member	Component or responsibility	Evidence of contribution
[Name 1]	Android Compose requester UI and navigation	Commits, screenshots, or work log
[Name 2]	Android donor setup, location, and Room persistence	Commits, migration test, or work log
[Name 3]	FastAPI routes, authorization, and contact lifecycle	Commits, test output, or work log
[Name 4]	PostgreSQL/PostGIS schema, deployment, and CI	Migration logs, CI runs, or work log
[Name 5]	Research, documentation, evidence, and integration testing	This report, test matrix, or work log

Appendix D — API and Data-Contract Reference

HTTP status conventions

Status	Meaning in LifeLink
200	Successful read or update
201	New emergency request, conversation, or message created
204	Push token accepted with no response body
400	Payload or lifecycle validation failure
401	Missing/invalid authentication
403	Authenticated but not authorized for the resource or state
404	Resource does not exist or is intentionally not disclosed
409	Idempotency or database conflict
429	Rate limit exceeded
503	Database/service unavailable

Matching states

```
awaiting_responses → cancelled  
awaiting_responses → fulfilled  
awaiting_responses → expired  
awaiting_responses → manual_broadcast
```

Contact states may include `pending`, `accepted`, `declined`, `arrived`, `contact_shared`, `meeting_arranged`, `fulfilled`, `cancelled`, and `expired`, subject to server transition rules.

Exact-location decision rule

```
show exact donor pin iff:  
  authenticated subject owns request  
  AND request is active  
  AND donor is a match/contact for request  
  AND contact status is accepted or later permitted state  
  AND donor.location_sharing_enabled is true  
  AND donor location/availability timestamp is fresh
```

Chat decision rule

```
allow chat iff:  
  authenticated subject is requester or donor participant  
  AND request/donor conversation exists or can be created  
  AND contact status is accepted or later permitted state  
  AND request is not cancelled, expired, or fulfilled
```

Final submission note

This document describes the repository as inspected on October 1, 2026. Replace all bracketed names, run the pending Android/PostgreSQL/live tests, attach evidence IDs, and update the status labels before submitting as a final academic report.